ХРАНИМЫЕ ПРОЦЕДУРЫ И ФУНКЦИИ

ТРиЗУБД



Часто при выполнении рутинных операций требуется производить одинаковые запросы. Хранимые процедуры позволяют объединять последовательность таких запросов и сохранить их на сервере. После этого пользователю не придётся посылать на сервер последовательность запросов, достаточно отправить один запрос на выполнение хранимой процедуры

PREPARED STATEMENTS (ГОТОВЫЕ ИНСТРУКЦИИ)

- Операторы обработки готовых инструкций PREPARE, EXECUTE, USING дают возможность создать инструкцию в формате строки и выполнить её
- Это можно сравнить с функцией eval() в PHP или JS, но с возможностью безопасной вставки данных



СИНТАКСИС PREPARE

PREPARE stmt_name FROM preparable_stmt

- **stmt_name** — название готовой инструкции.
- **stmt_name** не является уже строкой.
- **preparable_stmt** — инструкции в строковом формате.



СИНТАКСИС EXECUTE

EXECUTE stmt_name [USING @var_name [, @var_name] ...]

- Выполняет готовую инструкцию.
- Инструкция USING позволяет вставлять переменные в инструкцию.
- В этом случае в инструкции ставится «?», который при помощи USING заменятся на переменную.



```
PREPARE stmt1
from 'SELECT SQRT(POW(?,2) + POW(?,2)) AS `Гипотенуза`';
SET @a = 3;
SET @b = 4;
EXECUTE stmt1 USING @a, @b;
```



Result Grid

	Гипотенуза
▶	5

СИНТАКСИС DEALLOCATE

- {DEALLOCATE | DROP} PREPARE stmt_name
- Удаляет ранее созданную готовую инструкцию.



```
PREPARE stmt2
from 'select * from students where kom = ?';
set @c = 1;
EXECUTE stmt2 USING @c;
```

ПРИМЕР 2



ХРАНИМЫЕ ПРОЦЕДУРЫ И ФУНКЦИИ

- – это подпрограммы, которые могут быть многократно вызваны в **SQL**-запросах и/или из других процедур и функций, тогда, когда это понадобится.

или

- Хранимые процедуры представляют собой набор команд SQL, которые могут компилироваться и храниться на сервере.



ХРАНИМЫЕ ПРОЦЕДУРЫ ИМЕЮТ РЯД ПРЕИМУЩЕСТВ

- Повторное использование кода;
- Сокращение сетевого трафика;
- Безопасность (хранимая процедура гарантирует, что последовательность действий не приведёт к нарушению целостности данных);
- Простота доступа. Хранимые процедуры позволяют инкапсулировать сложный код и оформить его в виде простого вызова с осмысленным именем;
- Выполнение деловой логики: хранимые процедуры могут выполнять проверку условий выполнения заказа. Большая часть логики оформлена в виде хранимых процедур и не зависит от языка разработки внешнего приложения;



СОЗДАНИЕ ХРАНИМОЙ ПРОЦЕДУРЫ

CREATE

[DEFINER = { user | CURRENT_USER }]

PROCEDURE имя_процедуры ([список_параметров[,...]])

[характеристика]

тело_процедуры



DEFINER

- Необязательное предложение DEFINER определяет логин MySQL, который нужно использовать при проверке привилегий доступа для хранимого кода, которые имеют характеристику SQL SECURITY DEFINER
- По умолчанию **DEFINER=CURRENT_USER.**



ИМЯ ПРОЦЕДУРЫ

- Название хранимого кода. Символы допускаются такие же как и у переменных.



ПАРАМЕТРЫ

- список параметров, каждый из которых позволяет передать в процедуру или из неё входные данные или результат работы функции



[IN | OUT | INOUT] PARAM_NAME TYPE

- **IN** - данные передаются строго внутрь хранимой процедуры,;
- **OUT** - данные передаются строго из хранимой процедуры;
- **INOUT** – значение этого параметра как принимается во внимание внутри процедуры, так и сохраняет своё значение по выходу из неё.



IN

- данные передаются строго внутрь хранимой процедуры, но если параметру с данным модификатором внутри функции присваивается новое значение, по выходу из неё оно не сохраняется и параметр принимает значение, которое он имел до вызова процедуры;



OUT

- данные передаются строго из хранимой процедуры, даже если параметр имеет какое-нибудь начальное значение, внутри хранимой процедуры оно не принимается во внимание.
- Если параметр изменялся внутри процедуры, после вызова процедуры параметр имеет значение, присвоенное ему внутри процедуры



INOUT

- значение этого параметра как принимается во внимание внутри процедуры, так и сохраняет своё значение по выходу из неё



ПО УМОЛЧАНИЮ

- если ни один из модификаторов не указан, то считается, что параметр объявлен с ключевым словом **IN**.



- Список аргументов, заключённых в круглые скобки является обязательным, если они отсутствуют, то следует использовать пустой список аргументов ().



ХАРАКТЕРИСТИКА

- LANGUAGE SQL
  | [NOT] DETERMINISTIC
  | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
  | SQL SECURITY { DEFINER | INVOKER }
  | COMMENT 'string'



- Характеристики DETERMINISTIC и NOT DETERMINISTIC указывают, производит ли функция всегда тот же самый результат для входных данных.
- Значение по умолчанию: NOT DETERMINISTIC, если никакая характеристика не дана.



- Характеристики CONTAINS SQL, NO SQL, READS SQL DATA и MODIFIES SQL DATA обеспечивают информацию относительно того, читает ли функция или записывает данные.
- NO SQL или READS SQL DATA указывают, что функция не изменяет данные.
- Значение по умолчанию: CONTAINS SQL, если никакая характеристика не дана.



- Характеристика SQL SECURITY может применяться для определения, должна ли процедура выполняться с использованием привилегий пользователя, создающего эту процедуру, или привилегий пользователя, ее вызывающего.
- Значение по умолчанию – DEFINER.



- Конструкция COMMENT является расширением MySQL и может использоваться для описания хранимой процедуры.
- Такая информация отображается операторами SHOW CREATE PROCEDURE.



ПО УМОЛЧАНИЮ ХАРАКТЕРИСТИКА ИМЕЕТ СЛЕДУЮЩИЕ ЗНАЧЕНИЕ:

NOT DETERMINISTIC

CONTAINS SQL

SQL SECURITY DEFINER

COMMENT ''



ТЕЛО ПРОЦЕДУРЫ

- Состоит из допустимой инструкций хранимого кода SQL
  - Это может быть простая инструкция типа SELECT или INSERT , либо это может быть составная инструкция, использующая BEGIN и END.
  - Составные инструкции могут содержать объявления, циклы и другие инструкции управляющей структуры. Некоторые инструкции не допускаются в хранимых процедурах.



ВЫЗОВ ПРОЦЕДУРЫ

- CALL sp_name([parameter[,...]])



ПЕРЕМЕННЫЕ

- Для конкретного процесса пользователь может определить локальные переменные, которые в MySQL обозначаются как @variablename.
- Имя локальной переменной может состоять из буквенно-цифровых символов установленного в данное время алфавита и символов ‘_’, ‘$’, и ‘.’.



КОДИРОВКА

- Тип кодировки по умолчанию — **ISO-8859-1 Latin1**, он может быть изменен указанием иного типа в аргументе параметра — **default-character-set mysqld**.



- Все локальные переменные содержат значение **NULL** по умолчанию; в них могут храниться целые числа, вещественные числа или строковые величины.
- При запуске конкретного процесса все объявленные в нем локальные переменные автоматически активизируются.



ЛОКАЛЬНЫЕ ПЕРЕМЕННЫЕ ПРОЦЕДУРЫ

- Локальную переменную можно объявить, используя синтаксис команды SET:
- SET @variable=
  { integer expression
  | real expression
  | string expression }
  [,@variable= ...]



- Локальные переменные не требуют инициализации.
- Они содержат значение NULL по умолчанию; в них могут храниться целые числа, вещественные числа или строковые величины.
- При запуске конкретного процесса все объявленные в нем локальные переменные автоматически активизируются.



ПРИМЕР ОБЪЯВЛЕНИЯ ПЕРЕМЕННОЙ ЧЕРЕЗ SET:

- SET @myvar = "Hello, Aco!";
- SELECT @myvar;



SELECT @T1:=(@T2:=1)+@T3:=4, @T1, @T2, @T3;

- Можно также определить значение переменной иным способом, без команды SET.
- Однако в этом случае в качестве оператора присвоения более предпочтительно использовать оператор ‘:=’, чем оператор ‘=’, так как последний зарезервирован для сравнения выражений, не связанных с установкой переменных



- Введенные пользователем переменные могут применяться только в составе выражений и там, где выражения допустимы.
- В область их применения в данное время не включается контекст, в котором явно требуется число, например, условие LIMIT в команде SELECT или выражение IGNORE number LINES в команде LOAD DATA.



ПРЕИМУЩЕСТВА ИСПОЛЬЗОВАНИЯ ПЕРЕМЕННЫХ

- Возможность сохранять промежуточные результаты в переменных и оперировать им по ходу дальнейшего выполнения запроса;
- Позволяют значительно ускорить выполнение некоторых запросов (за счет меньшего количества сканирований таблицы),
- Позволяет выполнять такие запросы, которые в стандартной реляционной модели реализуются очень сложно или вовсе не реализуются.



НЕДОСТАТКИ ИСПОЛЬЗОВАНИЯ ПОЛЬЗОВАТЕЛЬСКИХ ПЕРЕМЕННЫХ

- Сложная переносимость на другие СУБД (механизм ПП служит определенной надстройкой над реляционной моделью баз данных в рамках СУБД MySQL).
- Поведение пользовательских переменных будет зависеть от порядка выполнения сложного запроса, который может меняться оптимизатором MySQL.



ОБЛАСТИ ВИДИМОСТИ

- Существуют два вида переменных: пользовательские переменные и переменные хранимого кода.
- Пользовательские переменные создаются в момент присвоения им значения. Это значение хранится в переменной до завершения соединения с сервером.



ПРИМЕР

- SET @var=123;
- SELECT @var;



ТИПЫ ПЕРЕМЕНЫХ

- Переменные хранимого кода же напротив — имеют ограниченную область видимости: в подпрограмме создавшей её.
- Они должны объявляться в хранимом коде и имеют совершенно определенный тип данных.
- MySQL сервер поддерживает два типа переменных, глобальные и сеансовые.
  - **Глобальные переменные** - инициализируются при старте MySQL сервера, получая значения по умолчанию.
  - **Сеансовые переменные** - создаются для каждого соединения клиента с сервером и получают значения, установленные для глобальных переменных



ВРЕМЯ ЖИЗНИ ПЕРЕМЕННЫХ

- Переменные хранимого кода живут до завершении процедуры/функции/триггера в которых они объявлены.
- Пользовательские переменные видимы только в текущей сессии



СИСТЕМНЫЕ ПЕРЕМЕННЫЕ

- специфичные для потока, которые уникальны для данного соединения
- глобальные переменные, которые предназначаются для управления глобальными событиями.



ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ

- Все глобальные переменные инициализируются из командной строки и файлов опций
- Можно изменять значения с помощью SET GLOBAL.
  - **SET GLOBAL read_buffer_size=16384\g**



СИСТЕМНЫЕ ПЕРЕМЕННЫЕ MYSQL СЕРВЕРА

- port порт, прослушиваемый MySQL сервером. Значение по умолчанию 3306
- **Version** содержит версию **MySQL** сервера

- https://vds-admin.ru/mysql/sistemnye-peremennye-mysql-servera-fail-mycnf



ТЕЛО ПРОЦЕДУРЫ

- Тело процедуры состоит из составного оператора **BEGIN** … **END**, внутри которого могут располагаться другие операторы, в т.ч. и другие составные операторы **BEGIN** … **END**.

[label:] **BEGIN**

statements

END [label]



МЕТКА

- Если оператор начинается с необязательной метки **label**, в качестве которой может выступать любое уникальное имя, то он может заканчиваться выражением **END label**



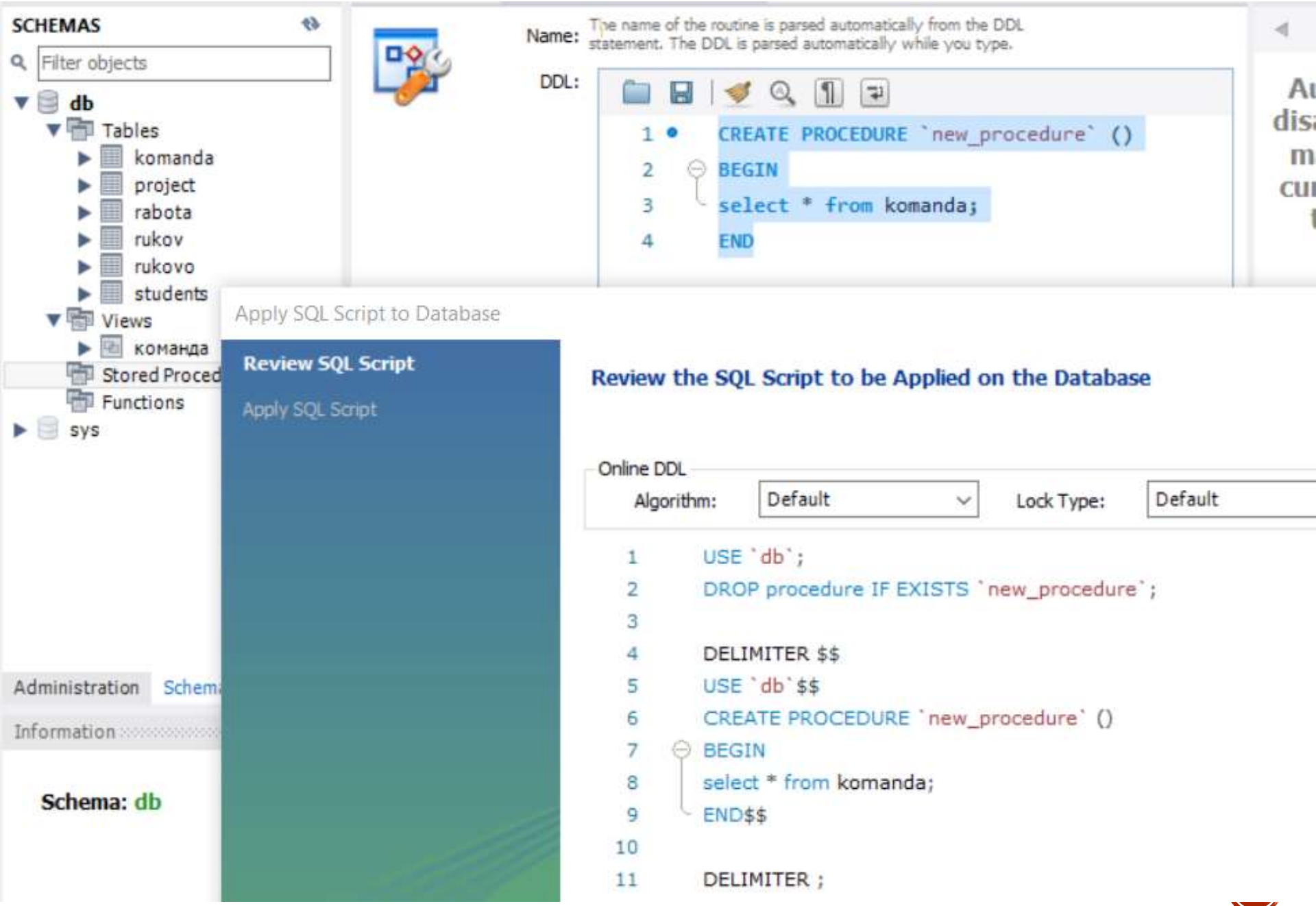
- Если хранимая процедура содержит только один запрос, то можно не использовать составной оператор **BEGIN … END**



ПЕРЕНАПРАВЛЕНИЕ СИМВОЛА ОКОНЧАНИЯ ЗАПРОСА

- Обязательный символ (;) в конце каждого запроса воспринимается консольным клиентом как сигнал отправки запроса на сервер.
- При работе с хранимыми процедурами следует переопределить разделитель запросов при помощи параметра **–delimiter=//** консольного клиента **mysql**.
- В этом случае для обозначения ввода вместо (;) следует использовать последовательность //.




SCHEMAS
Filter objects
db
Tables
komanda
project
rabota
rukov
rukovo
students
Views
команда
Stored Proced
Functions
sys
Name:
The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.
DDL:
1 CREATE PROCEDURE `new_procedure` ()
2 BEGIN
3 select * from komanda;
4 END
Apply SQL Script to Database
Review SQL Script
Apply SQL Script
Review the SQL Script to be Applied on the Database
Online DDL
Algorithm:
Default
Lock Type:
Default
1 USE `db`;
2 DROP procedure IF EXISTS `new_procedure`;
3
4 DELIMITER $$
5 USE `db`$$
6 CREATE PROCEDURE `new_procedure` ()
7 BEGIN
8 select * from komanda;
9 END$$
10
11 DELIMITER ;
Administration
Information
Schema: db

ПЕРЕНАПРАВЛЕНИЕ СИМВОЛА ОКОНЧАНИЯ ЗАПРОСА:

- **mysql -u root --delimiter=//**



ИМЯ ХРАНИМОЙ ПРОЦЕДУРЫ

- **не может превышать 64 символа и не зависит от регистра**



СИНТАКСИС ХРАНИМОГО КОДА

```
DELIMITER $$
CREATE PROCEDURE someproc()
BEGIN
  # ...
  SELECT 1;
  # ...
END$$

CALL someproc() # Выведет 1
```



КОНСТРУКЦИЯ BEGIN … END

[begin_label:] BEGIN

  [statement_list]

END [end_label]

- Хранимый код может включать множественные инструкции, благодаря составному оператору BEGIN…END, который и определяет блок инструкций.
- Значения begin_label и end_label, если оба заданы, должны быть одинаковыми. label позволяет не путать операторы, если они вложены.



ОПЕРАТОР DECLARE

- позволяет определить значение временной переменной, который имеет синтаксис:
- **Declare имя_перем тип_данных [default значение_по_умолчанию]**

- DECLARE a INT DEFAULT 5;



УСЛОВИЕ DECLARE CONDITION ДЛЯ ОБРАБОТЧИКА

- **DECLARE condition_name CONDITION FOR condition_value** *condition_value*:
- **SQLSTATE [VALUE] sqlstate_value | mysql_error_code** Эта инструкция определяет условия, которые нуждаются в специфической обработке. Она сопоставляет имя с определенным условием ошибки. Имя может впоследствии использоваться в инструкции **DECLARE HANDLER.** Вместе с **DECLARE HANDLER**(см. ниже) они образуют своеобразный **try — catch.**
- Здесь **condition_value** может быть значением **SQLSTATE** или же кодом ошибки **MySQL.**



КУРСОРЫ

- MySQL позволяет использовать курсоры (CURSORs) в хранимых процедурах и функциях.
- **Курсоры – это указатели на ресурс выборки.**
- Курсоры существуют только в хранимой процедуре или функции. Курсоры только на чтение, не могут быть «прокручены» и не могут быть клонированы.
- Курсоры крайне медленные, поэтому используйте их по необходимости.



- Курсоры должны быть объявлены до их использования. Переменные с условиями объявляются прежде курсоров.
- Обработчики объявляются строго после объявления курсоров.



СОЗДАЮТСЯ КУРСОРЫ ПРИ ПОМОЩИ ОПЕРАТОРА DECLARE

- DECLARE cursor_name CURSOR FOR select_statement



ПРИМЕР

//пара переменных для последующей выборки данных курсора

DECLARE a, b INT;

//объявляем курсор, выбирающий пары значений из некой тестовой таблицы

DECLARE cur1 CURSOR FOR

SELECT value_a, value_b FROM test.t1;



ОТКРЫТИЕ КУРСОРА

- Инструкция OPEN открывает предварительно объявленный курсор.
- **OPEN cursor_name**



ВЫБОРКА ИЗ КУРСОРА В ПЕРЕМЕННУЮ

FETCH cursor_name INTO var_name [, var_name] …

- Инструкция FETCH выбирает следующую строку (если строка существует), используя определенный открытый курсор, и продвигает указатель курсора.
- Если больше нет доступных строк, происходит условие No Data со значением SQLSTATE 02000. Чтобы обнаружить это условие, можно установить обработчик для этого.



ПРИМЕР

```
//помещаем следующую пару значений в переменные a и b
FETCH curl INTO a, b;
//выполняем какие то манипуляции с данными
IF a < b THEN
   …
ELSE
   …
END IF;
```



ЗАКРЫТИЕ КУРСОРА

CLOSE cursor_name

- Закрывает предварительно открытый курсор.
- Если курсор не закрыт, он все равно закроется в конце составной инструкции (обычно по наступлению END), в которой он был объявлен.
- Обратите внимание: с начало объявляются переменные, потом курсоры, а затем обработчик и условие.



КАК ИСПОЛЬЗОВАТЬ КУРСОРЫ

- При помощи оператора **DECLARE** объявляется курсор для отдельного оператора **SELECT** или для отдельной процедуры.
- Оператором **OPEN** производится открытие курсора.
- Используя оператор **FETCH**, осуществляется установление указателя на требуемую запись курсора. При этом значения полей текущей записи присваиваются переменным, указываемым в операторе **FETCH**. Обычно это конструкция помещается в цикл, который прерывается по некоторому условию.
- В процессе перемещения указателя текущей записи курсора при выходе указателя за пределы курсора устанавливается значение **SQLSTATE = 02000**.
- После того как курсор становится ненужным, он закрывается оператором **CLOSE**.



ПРИМЕР

```
CREATE PROCEDURE curdemo()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE a CHAR(16);
  DECLARE b,c INT;
  DECLARE cur1 CURSOR FOR SELECT id,data FROM test.t1;
  DECLARE cur2 CURSOR FOR SELECT i FROM test.t2;
  DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET done = 1;
  OPEN cur1;
  OPEN cur2;
    # что-нибудь делаем
  CLOSE cur1;
  CLOSE cur2;
END
```



КОНСТРУКЦИЯ SELECT … INTO

- Позволяет «заполнить» переменную, прямо из таблицы:
- SELECT col_name[,...] INTO var_name[,...] table_expr

Этот синтаксис SELECT сохраняет выбранные столбцы непосредственно в переменные.

Следовательно, количество выбранных столбцов слева от INTO должно быть равно количеству переменных справа от INTO .

- SELECT id,data INTO x,y FROM test.t1 WHERE id_tm = 5;



ОПЕРАТОРЫ УПРАВЛЕНИЯ ПОТОКОМ ДАННЫХ

- Оператор **IF…THEN…ELSE**
- Оператор множественного выбора **CASE**
- Оператор **WHILE**
- Оператор **REPEAT**
- Оператор **LOOP**
- Оператор **GOTO**



ОПЕРАТОР IF…THEN…ELSE

Оператор позволяет организовать ветвления в программе по условию и имеет следующий синтаксис:

IF условие THEN действия

[ELSEIF условие THEN действия]

[ELSE действия]

END IF

Логическое выражение (условие) принимает два значения: 0 (ложь) и 1 (истина).

Действия могут включать составной оператор BEGIN…END



ОПЕРАТОР МНОЖЕСТВЕННОГО ВЫБОРА CASE

1 форма: **CASE значение_выбора**

WHEN значение THEN действия

[WHEN значение THEN действия]
[ELSE действия]
END CASE

2 форма: **CASE**

WHEN условие THEN действия

[WHEN условие THEN действия]
[ELSE действия]
END CASE



ПРИМЕР

SELECT CASE 1 WHEN 1 THEN "one" WHEN 2 THEN "two" ELSE "more" END; # "one"

SELECT CASE WHEN 1>0 THEN "true" ELSE "false" END; # "true"

SELECT CASE BINARY "B" WHEN "a" THEN 1 WHEN "b" THEN 2 END; # NULL



ОПЕРАТОР WHILE

- Оператор While выполняет цикл и имеет следующий синтаксис:

[label:] WHILE условие DO

Действия

END WHILE [label]



ПРИМЕР: ПРОЦЕДУРУ, КОТОРАЯ ТРИ РАЗА ВЫВОДИТ ТЕКУЩУЮ ДАТУ С ПОМОЩЬЮ ФУНКЦИИ NOW()

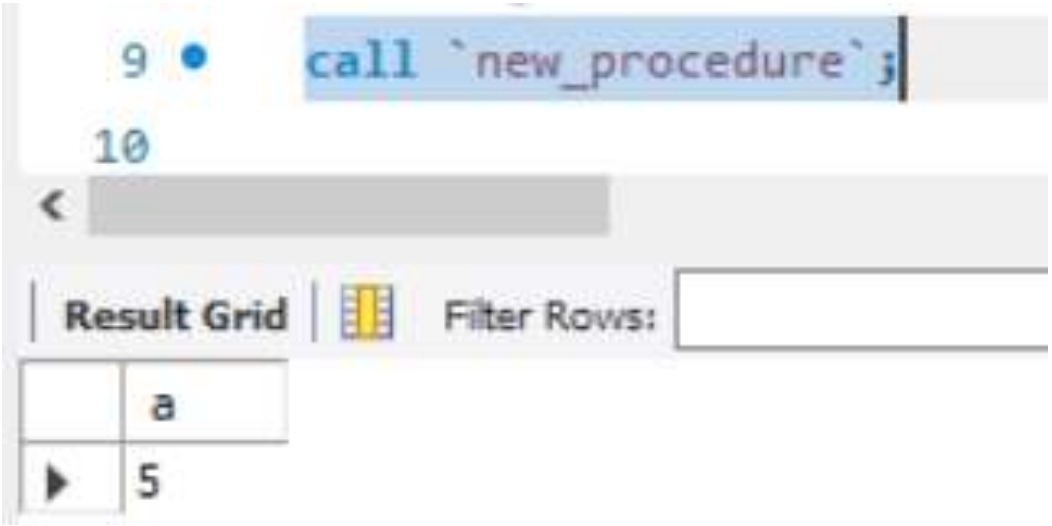
```
CREATE PROCEDURE  NOW3 ()
BEGIN
        DECLARE I INT DEFAULT 3;
While i>0 DO
SELECT NOW();
SET i=i-1;
END WHILE;
END
```



ПРИМЕР

```
CREATE PROCEDURE dowhile()
BEGIN
  DECLARE v1 INT DEFAULT 5;
  WHILE v1 > 0 DO
    SET v1 = v1 - 1;
  END WHILE;
END
```





ДЛЯ ВЫЗОВА ПРОЦЕДУРЫ

используется функция **CALL**

ДОСРОЧНЫЙ ВЫХОДА ИЗ ЦИКЛА

- Для досрочного выхода из цикла предназначен оператор **LEAVE**, который имеет синтаксис **LEAVE label;**



ИНСТРУКЦИЯ ITERATE

- ITERATE label ITERATE может появляться только внутри инструкций LOOP, REPEAT и WHILE.
- ITERATE означает «выполнить цикл снова».
- Можно сравнить с инструкцией continue из C++.



ОПЕРАТОР REPEAT

Оператор реализует цикл, так же как и оператор While:

[label:] REPEAT

Действия

UNTIL условие

END REPEAT [label]



ПРИМЕР

```
CREATE PROCEDURE dorepeat()
BEGIN
DECLARE p1 INT DEFAULT 1000;
SET @x = 0;
REPEAT SET @x = @x + 1;
  UNTIL @x > p1
END REPEAT;
END
```



ОПЕРАТОР LOOP

Оператор реализует цикл:

[label:] LOOP

Действия

END LOOP [label]



ПРИМЕР

```
CREATE PROCEDURE doiterate()
BEGIN
 DECLARE p1 INT DEFAULT 5;
 label1: LOOP
  SET p1 = p1 + 1;
  IF p1 < 10 THEN ITERATE label1; # если p1<10 повторяем цикл
  END IF;
  LEAVE label1;  # покидаем цикл
 END LOOP label1;
 SET @x = p1;
END
```



ОПЕРАТОР GOTO

- Оператор осуществляет безусловный переход и имеет следующий синтаксис:
- **GOTO label**



```
9 •  call `my_version`;
10
```

Result Grid | Filter Rows:

VERSION()
8.0.23

```
USE `db`;
DROP procedure IF EXISTS `my_version`;

DELIMITER $$
USE `db`$$
create procedure my_version()
begin
select VERSION();
end$$

DELIMITER ;
```

ПРИМЕР1: ПРОЦЕДУРА, КОТОРАЯ ВЫВОДИТ ИМЯ ВЕРСИИ MYSQL



ПРИМЕР2. ПРОЦЕДУРА, КОТОРАЯ ПОДСЧИТЫВАЕТ КОЛИЧЕСТВО ЗАПИСЕЙ В ТАБЛИЦЕ STUDENTS:

```
CREATE PROCEDURE numST (out total int)
BEGIN
  select count(*) into total from Students;
END
```



ПРИМЕР 3. СОЗДАДИМ ХРАНИМУЮ ПРОЦЕДУРУ, КОТОРАЯ ПО ПЕРВИЧНОМУ КЛЮЧУ КНИГИ KOD_GR ВЫДАЁТ НАЗВАНИЕ ГРУППЫ.

- Для решения задачи требуется определить параметр **kod** с атрибутом **IN**, а параметр **gr_nazv** с атрибутом **OUT**

```
CREATE PROCEDURE gr_nazv (in kod smallint, out gr_nazv char(5))
BEGIN
   select gruppa into gr_nazv From GRUPS
   where  kod_gr= kod;
END
```

ЗАДАЧА

- Создайте готовую инструкцию для ввода значений в таблицу **Students**, при этом используйте готовые инструкции



ХРАНИМЫЕ ФУНКЦИИ



СИНТАКСИС:

CREATE

[DEFINER = { user | CURRENT_USER }]

FUNCTION ([parameter[,...]])

RETURNS type

[characteristic...]

routine_body



ОТЛИЧИЯ ОТ ПРОЦЕДУРЫ

1. Функция, в отличие от процедуры может вызываться непосредственно, без использования оператора **CALL** и возвращает одно значение, которое подставляется в место вызова функции.
2. При объявлении параметров функции использование ключевых слов **IN**, **INOUT**, **OUT** недопустимо. Все параметры, передаваемые функции являются входными.



ПАРАМЕТРЫ ФУНКЦИИ

- **sp_name**
- **sp_parameter** [IN] param_name type
- **RETURNS**
- type



ПРИМЕР ФУНКЦИИ, КОТОРАЯ ВЫДАЁТ НА ЭКРАН СООБЩЕНИЕ «ПРИВЕТ, **ИМЯ!**». ДЛЯ ЭТОГО ОПИШЕМ ФУНКЦИЮ:

```
CREATE FUNCTION hello(name char(15)) RETURNS char(50)
begin
return concat('Hello, ',name, '!');
end
```

- Для вызова функции команда Select hello('World');



ФУНКЦИЯ ДЛЯ ВЫЧИСЛЕНИЯ ФАКТОРИАЛА ВВЕДЁННОГО ЧИСЛА

```
CREATE  FUNCTION factorial (n int) RETURNS int(11)
begin
declare i int default 1;
declare r int default 1;
while (i<=n) do
set r=r*i;
set i=i+1;
end while;
return r;
end
```



ПРИМЕР ФУНКЦИИ, ВОЗВРАЩАЮЩЕЙ 1, ЕСЛИ A<B, 0, ЕСЛИ A>=B

```
CREATE FUNCTION eq(IN a INT, IN b INT) RETURNS TINYINT(1)
  NOT DETERMINISTIC
  SQL SECURITY INVOKER
COMMENT 'Пример'
BEGIN
  IF a < b
    THEN RETURN 1;
    ELSE RETURN 0;
  END IF;
END
```



ВЫЗОВ ПРОЦЕДУРЫ
SELECT EQ(1,3); # ВЫВЕДЕТ 1

```
CREATE FUNCTION eq(IN a INT, IN b INT) RETURNS TINYINT(1)
BEGIN
  IF a < b
    THEN RETURN 1;
    ELSE RETURN 0;
  END IF;
END
```



ПРИМЕЧАНИЯ

- По умолчанию, функция или процедура связана с заданной по умолчанию базой данных. Чтобы сопоставить явно с другой базой данных, надо определять имя как db_name.sp_name, при её создании
- Если стандартное имя совпадает с именем встроенной функции SQL, мы должны использовать пробел между именем и следующей круглой скобкой при определении подпрограммы или происходит синтаксическая ошибка. Следует избегать многократно использовать имена существующих функций SQL для ваших функций или процедур.
- Список параметров, включенный внутри круглых скобок, должен всегда присутствовать. Если не имеется никаких параметров, должен использоваться пустой список параметров ().



МЕТАДАННЫЕ

- Существует четыре способа просмотреть данные, относящиеся к хранимым процедурам и функциям:
  - Операторы **SHOW PROCEDURE STATUS** / **SHOW FUNCTION STATUS** – для просмотра списка уже созданных хранимых процедур/функций
  - Операторы **SHOW CREATE PROCEDURE** / **SHOW CREATE FUNCTION** – выводит оператор при помощи которого была создана процедура/функция
  - Запрос **SELECT FROM mysql.proc** – получение информации о хранимых процедурах из системной базы данных
  - Запрос **SELECT FROM information_schema**



ОПЕРАТОРЫ SHOW PROCEDURE STATUS

```
25    Show procedure status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Db	Name	Type	Definer	Modified	Created	Security_type	Comment
db	gr_nazv	PROCEDURE	root@localhost	2021-04-12 21:06:34	2021-04-12 21:06:34	DEFINER	
db	my_version	PROCEDURE	root@localhost	2021-04-12 20:56:46	2021-04-12 20:56:46	DEFINER	
db	new_procedure	PROCEDURE	root@localhost	2021-04-12 20:50:39	2021-04-12 20:50:39	DEFINER	
sys	create_synonym_db	PROCEDURE	mysql.sys@localhost	2021-04-07 21:27:19	2021-04-07 21:27:19	INVOKER	Description -------
sys	diagnostics	PROCEDURE	mysql.sys@localhost	2021-04-07 21:27:19	2021-04-07 21:27:19	INVOKER	Description -------
sys	execute_prepared_stmt	PROCEDURE	mysql.sys@localhost	2021-04-07 21:27:19	2021-04-07 21:27:19	INVOKER	Description -------



```
1      delimiter //
2  •   CREATE PROCEDURE gr_nazv (in kod char(4), out pr_nazv varchar(25)
3      BEGIN
4          select nazv into pr_nazv From project
5          where  shifr = kod;
6      END //
7
8  •   Show CREATE procedure gr_nazv;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

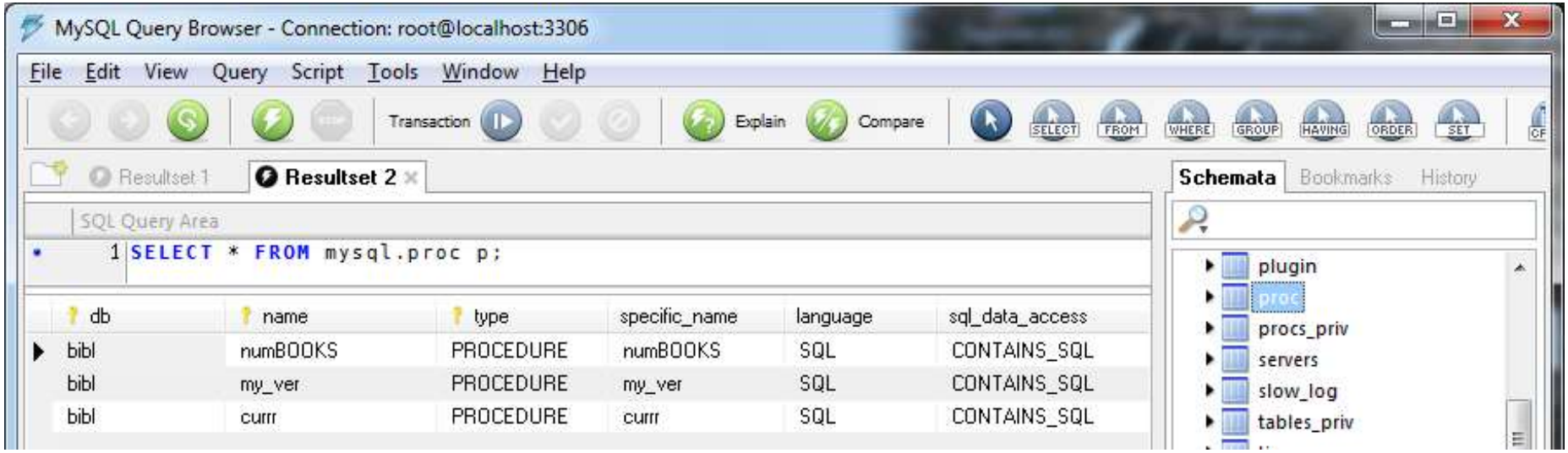
Procedure	sql_mode	Create Procedure
gr_nazv	STRICT_TRANS_TABLES,NO_ENGINE_SUBSTIT...	CREATE DEFINER=`root`

Result Grid

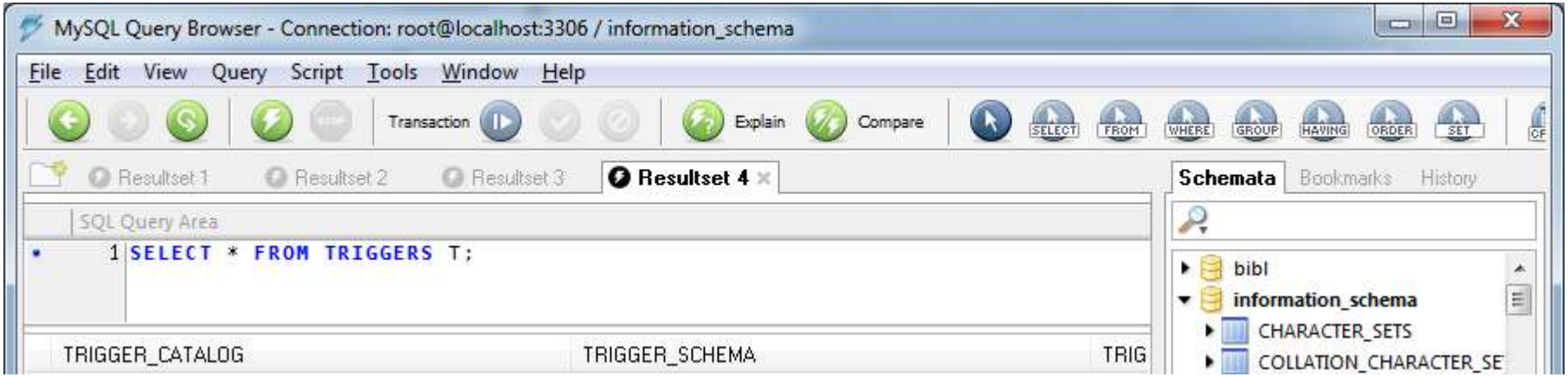
Form

SHOW CREATE PROCEDURE

SELECT FROM MYSQL.PROC



SELECT FROM INFORMATION_SCHEMA



УДАЛЕНИЕ ХРАНИМЫХ ПРОЦЕДУР / ФУНКЦИЙ

- Для удаления хранимых процедур/функций предназначен оператор
- **DROP PROCEDURE [if exists] имя_процедуры**



РЕДАКТИРОВАНИЕ ХРАНИМЫХ ПРОЦЕДУР/ФУНКЦИЙ

- Для изменения характеристик хранимой процедуры/функции предназначен оператор

```
ALTER {PROCEDURE | FUNCTION} sp_name
        [characteristic ...]

characteristic:
{CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
| SQL SECURITY { DEFINER | INVOKER }
| COMMENT 'string'
```



ОБРАБОТЧИКИ ОШИБОК

- Во время выполнения хранимых процедур/функций могут происходить самые разнообразные ошибки. Ошибки следует отслеживать и предпринимать действия, направленные на их предотвращение.
- СУБД MYSQL поддерживает обработчик ошибок, для объявления обработчика ошибок предназначен оператор DECLARE… HANDLER FOR:
- **DECLARE тип_обрабочика HANDLER FOR код_ошибки действия**



ТИП ОБРАБОТЧИКА МОЖЕТ ПРИНИМАТЬ ЗНАЧЕНИЕ:

- **CONTINUE** – выполнение текущей операции продолжается после выполнения операций обработчика
- **EXIT** – выполнение составного оператора **BEGIN…END**, в котором объявлен обработчик, прекращается



КОД ОШИБКИ МОЖЕТ БЫТЬ УКАЗАН ЧЕРЕЗ ЗАПЯТУЮ И МОЖЕТ ПРИНИМАТЬ ОДНО ИЗ СЛЕДУЮЩИХ ЗНАЧЕНИЙ:

- **SQLSTATE** – является пятисимвольным кодом ошибки в шестнадцатеричном формате и является стандартом в **SQL**
- **SQLWARNING** – любое предупреждение **MySQL**. Код ошибки начинается с ‘01’
- **NOT FOUND** – любая ошибка **MySQL**, связанная с невозможностью найти объект (таблицу, процедуру, функцию, столбец). Код ошибки начинается с **‘02’**
- **SQLEXCEPTION** – ошибки, не охваченные ключевыми словами **SQLWARNING** и **NOT FOUND**
- **mysql_error_code** – обычные четырёхзначные ошибки **MySQL**
- **condition_name** – имя условия, которое объявляется при помощи оператора **DECLARE…CONDITOR FOR**



ЗАДАНИЕ ДЛЯ ВЫПОЛНЕНИЯ

- Создайте выполняемую хранимую процедуру для вычисления суммы (произведения) натуральных чисел от 1 до n.
- Напишите функцию, которая по номеру месяца вывести выведет название месяца (например, январь).



ДЗ

Вам нужно реализовать 3 представления по вашей бд

1 из низ модифицируемое должно быть

